

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/py
```

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("Week5Assignment") \
    .getOrCreate()
```

```
print("Spark Session Created Successfully")
```

```
Spark Session Created Successfully
```

```
import os
```

```
os.listdir()
```

```
['.config', 'shopping_trends.csv', 'sample_data']
```

```
df = spark.read.csv(
    "shopping_trends.csv",
    header=True,
    inferSchema=True
)
```

```
print("Rows:", df.count())
print("Columns:", len(df.columns))
```

```
df.show(5)
```

```
Rows: 3900
Columns: 19
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
|Customer ID|Age|Gender|Item Purchased|Category|Purchase Amount (USD)|Lo
+-----+-----+-----+-----+-----+-----+-----+-----+
|          1| 55| Male|      Blouse|Clothing|          53|    Ke
|          2| 19| Male|      Sweater|Clothing|          64|
|          3| 50| Male|        Jeans|Clothing|          73|Massach
|          4| 21| Male|      Sandals|Footwear|          90| Rhode
|          5| 45| Male|      Blouse|Clothing|          49|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
df.printSchema()
```

```
root
|-- Customer ID: integer (nullable = true)
|-- Age: integer (nullable = true)
|-- Gender: string (nullable = true)
```

```
-- Item Purchased: string (nullable = true)
-- Category: string (nullable = true)
-- Purchase Amount (USD): integer (nullable = true)
-- Location: string (nullable = true)
-- Size: string (nullable = true)
-- Color: string (nullable = true)
-- Season: string (nullable = true)
-- Review Rating: double (nullable = true)
-- Subscription Status: string (nullable = true)
-- Payment Method: string (nullable = true)
-- Shipping Type: string (nullable = true)
-- Discount Applied: string (nullable = true)
-- Promo Code Used: string (nullable = true)
-- Previous Purchases: integer (nullable = true)
-- Preferred Payment Method: string (nullable = true)
-- Frequency of Purchases: string (nullable = true)
```

```
print(df.columns)
```

```
s', 'Payment Method', 'Shipping Type', 'Discount Applied', 'Promo Code Used',
```

```
print("Rows :", df.count())
print("Columns :", len(df.columns))
```

```
df.show(5)
```

```
Rows : 3900
```

```
Columns : 19
```

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location
1	55	Male	Blouse	Clothing	53	Ke
2	19	Male	Sweater	Clothing	64	
3	50	Male	Jeans	Clothing	73	Massach
4	21	Male	Sandals	Footwear	90	Rhode
5	45	Male	Blouse	Clothing	49	

```
only showing top 5 rows
```

```
from pyspark.sql.functions import col, sum
```

```
df.select([
    sum(col(c).isNull().cast("int")).alias(c)
    for c in df.columns
]).show()
```

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location
0	0	0	0	0	0	

```
print("Total Rows :", df.count())
print("Unique Rows :", df.dropDuplicates().count())
```

```
Total Rows : 3900
Unique Rows : 3900
```

Q1. What are the key limitations of traditional MapReduce that make Spark

```
# answer que1 : -> MapReduce writes intermediate data to disk after every st
#It is also not very efficient for iterative tasks like machine learning and
#which makes processing much faster. Spark also provides easy APIs for SQL,
```

Q2. Explain how Spark uses In-Memory Computing to speed up iterative mach

```
# Spark stores intermediate data in memory instead of writing it to disk aft
# repeated calculations in machine learning algorithms become much faster. I
# which increases execution time.
```

Q3: Write a code snippet to remove all duplicate rows from a DataFrame bas

```
# removing duplicate rows using Customer ID
```

```
df2 = df.dropDuplicates(["Customer ID"])
```

```
print("before :", df.count())
print("after  :", df2.count())
```

```
before : 3900
after  : 3900
```

```
df.select("Location").distinct().show()
```

```
+-----+
| Location|
+-----+
|      Utah|
|    Hawaii|
| Minnesota|
|      Ohio|
|    Oregon|
|  Arkansas|
|    Texas |
|North Dakota|
|Pennsylvania|
|Connecticut|
|    Vermont|
|   Nebraska|
|    Nevada|
| Washington|
|   Illinois|
|   Oklahoma|
|   Delaware|
|    Alaska|
| New Mexico|
|West Virginia|
```

```
+-----+
only showing top 20 rows
```

```
# Q4: Given a DataFrame df_sales, write a query to filter for rows where the
# then group by product_category to find the average sale_amount.
```

```
# NOTE : IN THE DATASET I HAVE CHOSEN NO WEST REGION IS THERE I HAVE CHOOSE
```

```
from pyspark.sql.functions import avg
```

```
df.filter(df["Location"] == "California") \
    .groupBy("Category") \
    .agg(avg("Purchase Amount (USD)").alias("avg_purchase")) \
    .show()
```

```
+-----+-----+
| Category| avg_purchase|
+-----+-----+
| Outerwear|59.66666666666664|
| Clothing|58.297872340425535|
| Footwear| 60.18181818181818|
| Accessories|59.516129032258064|
+-----+-----+
```

```
## Q5. Difference between .na.drop() and .na.fill()
```

```
# .na.drop() removes rows that contain null values.
```

```
# .na.fill() replaces null values with a given value instead of removing the
```

```
# Example:The following code shows how null values can be replaced with 'Unk
```

```
df.select("Subscription Status").distinct().show()
```

```
+-----+
|Subscription Status|
+-----+
| No|
| Yes|
+-----+
```

```
# filling null values in Subscription Status column
```

```
df.na.fill({"Subscription Status": "Unknown"}).show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|Customer ID|Age|Gender|Item Purchased|Category|Purchase Amount (USD)|Lo
+-----+-----+-----+-----+-----+-----+-----+
|1|55|Male|Blouse|Clothing|53|Ke
|2|19|Male|Sweater|Clothing|64|
|3|50|Male|Jeans|Clothing|73|Massach
|4|21|Male|Sandals|Footwear|90|Rhode
|5|45|Male|Blouse|Clothing|49|
```

```
+-----+-----+
only showing top 5 rows
```

Q6. Write a query to find the total count of records for each city in a D

```
city_count = df.groupBy("Location").count()
```

```
city_count.filter(city_count["count"] > 50).show()
```

#Since no location had more than 100 records in this dataset,
#a threshold of 50 was used to demonstrate the grouping and filtering operat

```
+-----+-----+
|      Location|count|
+-----+-----+
|         Utah|    71|
|        Hawaii|    65|
|    Minnesota|    88|
|         Ohio|    77|
|        Oregon|    74|
|    Arkansas|    79|
|         Texas|    77|
| North Dakota|    83|
| Pennsylvania|    74|
| Connecticut|    78|
|        Vermont|    85|
|    Nebraska|    87|
|        Nevada|    87|
| Washington|    73|
|    Illinois|    92|
|    Oklahoma|    75|
|    Delaware|    86|
|        Alaska|    72|
| New Mexico|    81|
| West Virginia|    81|
+-----+-----+
only showing top 20 rows
```

Q7. How does the immutability of Spark DataFrames affect data cleaning?

#answer 7 -> Spark DataFrames are immutable, which means we cannot change th
Every operation like dropping a column or renaming it creates a new DataFr
This helps in maintaining data consistency and avoiding accidental changes

Q8. Filter rows where age is between 18 and 30 and subscription is Premium

```
df.filter((df["Age"] >= 18) &
          (df["Age"] <= 30) &
          (df["Subscription Status"] == "Yes")) \
    .show()
```

```
+-----+-----+-----+-----+-----+-----+
|Customer ID|Age|Gender|Item Purchased|    Category|Purchase Amount (USD)|
+-----+-----+-----+-----+-----+-----+
|          2| 19|  Male|      Sweater|    Clothing|                64|
```

4	21	Male	Sandals	Footwear	90	Rho
8	27	Male	Shorts	Clothing	34	
9	26	Male	Coat	Outerwear	97	West
12	30	Male	Shorts	Clothing	68	
17	25	Male	Sunglasses	Accessories	36	
21	21	Male	Pants	Clothing	51	
25	18	Male	Jacket	Outerwear	22	
26	18	Male	Hoodie	Clothing	25	
39	29	Male	Dress	Clothing	37	
43	20	Male	Coat	Outerwear	100	
44	25	Male	Scarf	Accessories	69	
54	29	Male	Skirt	Clothing	24	Mi
58	21	Male	Coat	Outerwear	64	West
63	30	Male	Sneakers	Footwear	21	Mi
71	22	Male	Belt	Accessories	29	
73	18	Male	Dress	Clothing	26	
78	22	Male	Dress	Clothing	53	
81	19	Male	Sandals	Footwear	72	
83	19	Male	Pants	Clothing	28	

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```
df.select("Subscription Status").distinct().show()
```

```
+-----+
|Subscription Status|
+-----+
|                No|
|                Yes|
+-----+
```

Q9. Why should null values be handled before aggregation?

If null values are not handled properly, the result of calculations like a
Cleaning null values before aggregation gives more accurate and reliable r

#Q10: Write the code to revise a column named raw_timestamp by casting it to

```
from pyspark.sql.functions import current_timestamp
from pyspark.sql.types import TimestampType
```

```
df_time = df.withColumn("raw_timestamp", current_timestamp())
```

```
df_time = df_time.withColumn(
    "event_time",
    df_time["raw_timestamp"].cast(TimestampType())
)
```

```
df_time = df_time.drop("raw_timestamp")
```

```
df_time.select("event_time").show(5)
```

```
+-----+
|event_time|
```

```
+-----+
|2026-06-28 20:16:...|
|2026-06-28 20:16:...|
|2026-06-28 20:16:...|
|2026-06-28 20:16:...|
|2026-06-28 20:16:...|
+-----+
only showing top 5 rows
```

```
df.select("Subscription Status").distinct().show()
```

```
+-----+
|Subscription Status|
+-----+
|                    No|
|                    Yes|
+-----+
```

Q11. Explain the Shuffle process in Spark.

#ANSWER -> Shuffle happens when data needs to move between different partitions
 # It is called a wide transformation because data from many partitions is combined
 # Shuffle can increase execution time, so it should be used carefully on large datasets

Q12: Write a code snippet that identifies and removes rows where the email is null

```
data2 = [
    ("amit@gmail.com", "amit"),
    (None, "rahul"),
    ("test@gmail.com", ""),
    ("abc@gmail.com", "abc")
]

df2 = spark.createDataFrame(data2, ["email", "username"])

df2.filter((df2["email"].isNotNull()) &
           (df2["username"] != "")).show()
```

```
+-----+-----+
|email|username|
+-----+-----+
|amit@gmail.com|amit|
|abc@gmail.com|abc|
+-----+-----+
```

#Q13: How do you use the .agg() function to calculate multiple statistics at once

```
from pyspark.sql.functions import min, max, mean
```

```
df.agg(
    min("Purchase Amount (USD)").alias("min_price"),
    max("Purchase Amount (USD)").alias("max_price"),
```

```
mean("Purchase Amount (USD)").alias("avg_price")
).show()
```

```
+-----+-----+-----+
|min_price|max_price|      avg_price|
+-----+-----+-----+
|      20|      100|59.76435897435898|
+-----+-----+-----+
```

Q14. What is the risk of using inferSchema=true on messy data?

#ANSWER -> If the data contains inconsistent formats, Spark may assign the w
For example, different date formats in the same column can create errors w
It is better to define the schema manually for important projects.

Q15: Write a final processing pipeline that:

1)Filters out duplicates.

2)Fills null prices with 0.

3)Groups by store_id to calculate total revenue.

#Note: The dataset does not contain a store_id column, so Location is used f

```
from pyspark.sql.functions import sum
```

```
final_df = df.dropDuplicates()
```

```
final_df = final_df.na.fill({"Purchase Amount (USD)": 0})
```

```
final_df.groupBy("Location") \
    .agg(sum("Purchase Amount (USD)").alias("total_revenue")) \
    .show()
```

```
+-----+-----+
|      Location|total_revenue|
+-----+-----+
|      Utah|      4443|
|     Hawaii|      3752|
|   Minnesota|      4977|
|      Ohio|      4649|
|     Oregon|      4243|
|   Arkansas|      4828|
|      Texas|      4712|
| North Dakota|      5220|
| Pennsylvania|      4926|
| Connecticut|      4226|
|    Nebraska|      5172|
|    Vermont|      4860|
|    Nevada|      5514|
| Washington|      4623|
|   Illinois|      5617|
|   Oklahoma|      4376|
|   Delaware|      4758|
```



```
|      Alaska|      4867|  
| New Mexico|      5014|  
|West Virginia|      5174|  
+-----+-----+  
only showing top 20 rows
```

Insights

- # - Spark DataFrames make data cleaning and aggregation easy.
- # - Removing duplicates improves data quality.
- # - Handling null values before calculations gives better results.
- # - GroupBy operations involve shuffle and may take more time on large datasets.
- # - In-memory processing is one of the main reasons Spark is faster than MapReduce.

Conclusion

In this assignment, different Spark DataFrame operations were performed such as
handling null values, removing duplicates and grouping data. The shopping trip data
data cleaning and transformation concepts. The results show how Spark can efficiently